GenSpark

Git workflows and PR hygiene

Git is the tool that lets teams work on the same codebase without stepping on each other's changes. Knowing the commands is only step one — the real skill is understanding the workflow conventions that keep a shared codebase sane as multiple people push changes into it every day. This article covers the core workflow patterns and what makes a pull request (PR) good versus a liability.

Why Workflow Conventions Matter

Without agreed-upon conventions, a shared repository quickly becomes chaotic: people overwrite each other's work, nobody can tell what a change was for, and untested code ends up mixed in with stable code. Git workflow conventions exist to keep changes isolated, reviewable, and safe to merge.

Branching Basics

The `main` (or `master`) branch typically represents the stable, deployable version of the code. You should never work directly on `main`. Instead, for any new piece of work — a feature, a bug fix, an experiment — you create a new branch off of `main`.

A common naming convention looks like:

```
feature/add-login-validation
bugfix/expense-form-crash
chore/update-dependencies
```

The prefix (`feature`, `bugfix`, `chore`) tells anyone scanning the branch list what kind of change it contains, and the rest of the name briefly describes what it does.

A Typical Workflow

1. **Pull the latest changes** from `main` before starting new work, so your branch is based on the most current code.

2. **Create a new branch** for your specific task.
3. **Make your changes**, committing along the way with clear, descriptive commit messages.
4. **Push your branch** to the remote repository.
5. **Open a pull request** to merge your branch into `main`.
6. **Get it reviewed** by at least one other engineer.
7. **Address feedback**, push updates to the same branch.
8. **Merge** once approved, then delete the branch.

This cycle repeats for every discrete piece of work, which is why keeping branches small and focused matters — a branch that tries to do five unrelated things at once is much harder to review and much riskier to merge.

Writing Good Commit Messages

A commit message should explain what changed and, when it's not obvious, why. Compare:

```
fix stuff
```

to:

```
Fix null pointer error when expense form is submitted without a date
```

The second tells anyone looking at the project history exactly what problem existed and what this commit addressed, without needing to open the diff.

What Makes a Good Pull Request

A pull request is a request for another engineer to review and approve your change before it becomes part of the shared codebase. Good PR hygiene makes that review fast and effective.

Keep it small. A PR that changes 15 files across three unrelated features is exhausting to review properly, and reviewers under time pressure will often just rubber-stamp it — which defeats the purpose of review. A focused PR that does one thing is easier to review carefully and easier to revert if

something goes wrong.

Write a clear description. At minimum, explain what the change does and why it was needed. If it fixes a bug, describe the bug. If it's tied to a ticket or requirement, reference it.

Self-review before requesting review. Read your own diff before asking someone else to. You'll often catch leftover debug code, commented-out blocks, or small mistakes yourself, saving your reviewer's time.

Respond to feedback constructively. Reviewers may ask questions or request changes. This is a normal, healthy part of the process, not a judgment of your ability. Address feedback directly rather than defensively.

Common Pitfalls

- **Committing directly to `main`.** This bypasses review entirely and risks introducing untested or broken code into the stable branch.
- **Massive, infrequent commits.** Committing once at the end of a multi-day task loses the ability to track incremental progress and makes it harder to isolate where a bug was introduced.
- **Vague commit messages** like "fix," "update," or "changes" that give future readers no information.
- **Leaving PRs open indefinitely.** Long-lived branches drift further from `main` over time, increasing the chance of painful merge conflicts.

Comprehension Check

1. Why should engineers avoid committing directly to `main`?
2. What's the tradeoff between opening one large PR covering several changes versus several small, focused PRs?
3. What should you do before requesting a review on your own PR, and why does it help?

← Previous: Coding best practices

Next: Linux/CLI; Docker basics; CI/CD Introduction →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!